

727723EUA1109 : Sanjay M

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

X, y = make_classification(n_samples=1200, n_features=25, n_informative=18, n_re

# 70% train, 15% val, 15% test
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50,

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

print(f"Train: {X_train.shape}, Val: {X_val.shape}, Test: {X_test.shape}")
```

Train: (840, 25), Val: (180, 25), Test: (180, 25)

```
In [2]: def build_model(input_dim):
    model = Sequential([
        # Layer 1
        Dense(256, activation='relu', input_shape=(input_dim,)),
        BatchNormalization(),
        Dropout(0.3),

        # Layer 2
        Dense(128, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),

        # Layer 3
        Dense(64, activation='relu'),
        BatchNormalization(),
        Dropout(0.3),

        # Layer 4
        Dense(32, activation='relu'),
        BatchNormalization(),
        Dropout(0.2),

        # Output
        Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=Adam(learning_rate=1e-3),
        loss='binary_crossentropy',
        metrics=['accuracy'])
```

```

    )
    return model

model = build_model(X_train.shape[1])
model.summary()

```

c:\Users\HP\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\dense.py:92: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	6,656
batch_normalization (BatchNormalization)	(None, 256)	1,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
batch_normalization_1 (BatchNormalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
batch_normalization_2 (BatchNormalization)	(None, 64)	256
dropout_2 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 32)	2,080
batch_normalization_3 (BatchNormalization)	(None, 32)	128
dropout_3 (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 1)	33



Total params: 51,841 (202.50 KB)

Trainable params: 50,881 (198.75 KB)

Non-trainable params: 960 (3.75 KB)

```

In [3]: early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights
        reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)

        history = model.fit(
            X_train, y_train,
            validation_data=(X_val, y_val),
            epochs=200,
            batch_size=32,

```

```
callbacks=[early_stop, reduce_lr],
verbose=1
)

stopped_epoch = early_stop.stopped_epoch
best_epoch    = stopped_epoch - early_stop.patience + 1
print(f"EarlyStopping triggered at epoch {stopped_epoch+1}, best epoch ~ {best_e
```

Epoch 1/200
27/27  4s 19ms/step - accuracy: 0.6250 - loss: 0.7542 - val_accuracy: 0.7722 - val_loss: 0.6261 - learning_rate: 0.0010

Epoch 2/200
27/27  0s 7ms/step - accuracy: 0.7321 - loss: 0.5415 - val_accuracy: 0.8000 - val_loss: 0.5547 - learning_rate: 0.0010

Epoch 3/200
27/27  0s 7ms/step - accuracy: 0.7810 - loss: 0.4827 - val_accuracy: 0.8667 - val_loss: 0.4825 - learning_rate: 0.0010

Epoch 4/200
27/27  0s 7ms/step - accuracy: 0.8405 - loss: 0.3734 - val_accuracy: 0.8833 - val_loss: 0.4250 - learning_rate: 0.0010

Epoch 5/200
27/27  0s 7ms/step - accuracy: 0.8440 - loss: 0.3573 - val_accuracy: 0.9167 - val_loss: 0.3638 - learning_rate: 0.0010

Epoch 6/200
27/27  0s 9ms/step - accuracy: 0.8726 - loss: 0.3079 - val_accuracy: 0.9111 - val_loss: 0.3119 - learning_rate: 0.0010

Epoch 7/200
27/27  0s 7ms/step - accuracy: 0.8881 - loss: 0.2822 - val_accuracy: 0.9278 - val_loss: 0.2573 - learning_rate: 0.0010

Epoch 8/200
27/27  0s 7ms/step - accuracy: 0.8929 - loss: 0.2607 - val_accuracy: 0.9444 - val_loss: 0.2191 - learning_rate: 0.0010

Epoch 9/200
27/27  0s 7ms/step - accuracy: 0.8976 - loss: 0.2421 - val_accuracy: 0.9389 - val_loss: 0.1942 - learning_rate: 0.0010

Epoch 10/200
27/27  0s 6ms/step - accuracy: 0.9155 - loss: 0.2137 - val_accuracy: 0.9444 - val_loss: 0.1746 - learning_rate: 0.0010

Epoch 11/200
27/27  0s 6ms/step - accuracy: 0.9119 - loss: 0.2019 - val_accuracy: 0.9611 - val_loss: 0.1572 - learning_rate: 0.0010

Epoch 12/200
27/27  0s 6ms/step - accuracy: 0.9143 - loss: 0.2049 - val_accuracy: 0.9556 - val_loss: 0.1550 - learning_rate: 0.0010

Epoch 13/200
27/27  0s 7ms/step - accuracy: 0.9345 - loss: 0.1782 - val_accuracy: 0.9500 - val_loss: 0.1393 - learning_rate: 0.0010

Epoch 14/200
27/27  0s 7ms/step - accuracy: 0.9393 - loss: 0.1892 - val_accuracy: 0.9500 - val_loss: 0.1401 - learning_rate: 0.0010

Epoch 15/200
27/27  0s 6ms/step - accuracy: 0.9333 - loss: 0.2088 - val_accuracy: 0.9500 - val_loss: 0.1387 - learning_rate: 0.0010

Epoch 16/200
27/27  0s 7ms/step - accuracy: 0.9393 - loss: 0.1716 - val_accuracy: 0.9556 - val_loss: 0.1349 - learning_rate: 0.0010

Epoch 17/200
27/27  0s 6ms/step - accuracy: 0.9429 - loss: 0.1664 - val_accuracy: 0.9556 - val_loss: 0.1434 - learning_rate: 0.0010

Epoch 18/200
27/27  0s 7ms/step - accuracy: 0.9345 - loss: 0.1732 - val_accuracy: 0.9500 - val_loss: 0.1419 - learning_rate: 0.0010

Epoch 19/200
27/27  0s 6ms/step - accuracy: 0.9464 - loss: 0.1443 - val_accuracy: 0.9500 - val_loss: 0.1393 - learning_rate: 0.0010

Epoch 20/200
27/27  0s 6ms/step - accuracy: 0.9417 - loss: 0.1323 - val_accuracy: 0.9556 - val_loss: 0.1254 - learning_rate: 0.0010

Epoch 21/200
27/27  0s 8ms/step - accuracy: 0.9500 - loss: 0.1351 - val_accuracy: 0.9500 - val_loss: 0.1240 - learning_rate: 0.0010
Epoch 22/200
27/27  0s 6ms/step - accuracy: 0.9524 - loss: 0.1183 - val_accuracy: 0.9556 - val_loss: 0.1231 - learning_rate: 0.0010
Epoch 23/200
27/27  0s 7ms/step - accuracy: 0.9607 - loss: 0.1167 - val_accuracy: 0.9556 - val_loss: 0.1283 - learning_rate: 0.0010
Epoch 24/200
27/27  0s 7ms/step - accuracy: 0.9583 - loss: 0.1230 - val_accuracy: 0.9556 - val_loss: 0.1489 - learning_rate: 0.0010
Epoch 25/200
27/27  0s 6ms/step - accuracy: 0.9464 - loss: 0.1245 - val_accuracy: 0.9444 - val_loss: 0.1421 - learning_rate: 0.0010
Epoch 26/200
27/27  0s 7ms/step - accuracy: 0.9524 - loss: 0.1182 - val_accuracy: 0.9556 - val_loss: 0.1309 - learning_rate: 0.0010
Epoch 27/200
27/27  0s 8ms/step - accuracy: 0.9738 - loss: 0.0832 - val_accuracy: 0.9556 - val_loss: 0.1439 - learning_rate: 0.0010
Epoch 28/200
27/27  0s 6ms/step - accuracy: 0.9607 - loss: 0.0933 - val_accuracy: 0.9556 - val_loss: 0.1468 - learning_rate: 5.0000e-04
Epoch 29/200
27/27  0s 7ms/step - accuracy: 0.9643 - loss: 0.1065 - val_accuracy: 0.9556 - val_loss: 0.1364 - learning_rate: 5.0000e-04
Epoch 30/200
27/27  0s 7ms/step - accuracy: 0.9714 - loss: 0.0817 - val_accuracy: 0.9500 - val_loss: 0.1397 - learning_rate: 5.0000e-04
Epoch 31/200
27/27  0s 12ms/step - accuracy: 0.9726 - loss: 0.0763 - val_accuracy: 0.9500 - val_loss: 0.1413 - learning_rate: 5.0000e-04
Epoch 32/200
27/27  0s 7ms/step - accuracy: 0.9798 - loss: 0.0792 - val_accuracy: 0.9556 - val_loss: 0.1313 - learning_rate: 5.0000e-04
EarlyStopping triggered at epoch 32, best epoch ~ 22

```
In [4]: # Metrics
y_pred = (model.predict(X_test) >= 0.5).astype(int).flatten()
print(classification_report(y_test, y_pred, target_names=['Class 0', 'Class 1']))

epochs_range = range(1, len(history.history['loss']) + 1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss curve
axes[0].plot(epochs_range, history.history['loss'], label='Train Loss')
axes[0].plot(epochs_range, history.history['val_loss'], label='Val Loss')
axes[0].axvline(x=best_epoch, color='red', linestyle='--', label=f'EarlyStopping')
axes[0].set_title('Loss Curves')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()

# Accuracy curve
axes[1].plot(epochs_range, history.history['accuracy'], label='Train Accuracy')
axes[1].plot(epochs_range, history.history['val_accuracy'], label='Val Accuracy')
axes[1].axvline(x=best_epoch, color='red', linestyle='--', label=f'EarlyStopping')
axes[1].set_title('Accuracy Curves')
```

```

axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()

plt.tight_layout()
plt.show()

```

6/6 ————— 0s 25ms/step

	precision	recall	f1-score	support
Class 0	0.95	0.95	0.95	86
Class 1	0.96	0.96	0.96	94
accuracy			0.96	180
macro avg	0.96	0.96	0.96	180
weighted avg	0.96	0.96	0.96	180

